Week 2 · Task 4: Linux Basics and Log Files (/var/log)

Name: thato
Programme: Cybersecurity internship (SOC Analyst track)
Environment: [Ubuntu / Kali VM, version, VirtualBox lab from Day 1]
Date: [date of practical]

> Note: items in [square brackets] are for you to replace with your own output or screenshots before submitting.

1. Introduction

Log files are the primary evidence a SOC analyst works with. Every login, privilege escalation, service start and error on a Linux system is written to a log, usually under `/var/log`. Being able to navigate the file system, read these files and filter them quickly with tools such as `grep` and `tail` is the foundation of detecting events like brute-force attacks, unauthorised sudo use and suspicious service activity.

In this task I practised core Linux commands, explored `/var/log`, searched `auth.log` for failed logins, and monitored logs live with `tail -f`. I also completed the TryHackMe rooms *Linux Fundamentals Part 1* and *Intro to Logs*.

2. Basic Linux Commands Practised

Command	Purpose	Example used
`pwd`	Show current directory	`pwd`
`ls -la`	List files with permissions, owners, hidden files	`ls -la /var/log`
`cd`	Change directory	`cd /var/log`

`cat`	Print a whole file	`cat /etc/os-release`
`less`	Page through a large file (`/word` to search, `q` to quit)	`sudo less /var/log/syslog`
`head` / `tail`	Show first or last lines	`tail -n 20 /var/log/syslog`
`tail -f`	Follow a file live as new lines arrive	`sudo tail -f /var/log/auth.log`
`grep`	Search for text patterns	`sudo grep 'Failed' /var/log/auth.log`
`wc -l`	Count lines	`... | wc -l`

[Screenshot 1: terminal showing pwd, ls -la and cd /var/log]

3. Important Log File Locations in /var/log

Log file	What it records	SOC use
`/var/log/auth.log`	Logins, SSH, sudo, su, authentication events (Debian/Ubuntu)	Detect brute force, unauthorised privilege use
`/var/log/secure`	Same as auth.log on RHEL/CentOS	Same

`/var/log/syslog`	General system messages from services and the OS	Timeline of system activity
`/var/log/kern.log`	Kernel messages	Driver, hardware and kernel-level issues
`/var/log/dpkg.log`	Package installs and removals	Spot unexpected software installs
`/var/log/apache2/` or `/var/log/nginx/`	Web server access and error logs	Detect web attacks and scanning
`/var/log/wtmp`	Successful login history (read with `last`)	Who logged in and when
`/var/log/btmp`	Failed login history (read with `lastb`)	Brute-force evidence
`/var/log/faillog`	Failed login counters per user	Quick failed-login overview

[Screenshot 2: output of ls -lh /var/log]

Note on systemd systems: [If auth.log was missing on your VM, say so here.] On some modern distributions, including Kali, logs are held in the systemd journal instead. In that case they are read with `journalctl` (for example `sudo journalctl -u ssh`), or `rsyslog` can be installed to recreate the traditional text files.

4. Reading auth.log and syslog

I opened both files with `less`:

sudo less /var/log/auth.log
sudo less /var/log/syslog

A typical authentication log line contains these fields:

Mon DD HH:MM:SS hostname service[pid]: message

For example, an illustrative failed SSH line looks like this:

Oct 23 14:05:12 ubuntu sshd[1234]: Failed password for invalid user admin from 192.168.56.101 port 52144 ssh2

- **Timestamp:** when the event happened
- **Hostname:** which machine logged it
- **Service[pid]:** the process that wrote it (here sshd)
- **Message:** outcome, username, source IP and port

[Screenshot 3: auth.log opened in less]

5. Searching for Failed Logins with grep

To generate test data, I made several deliberately incorrect login attempts against my own lab VM (for example `ssh wronguser@localhost` with bad passwords, and failed `su` attempts). I then searched the log.

Find all failed entries:

sudo grep 'Failed' /var/log/auth.log

Case-insensitive search for failed passwords:

sudo grep -i 'failed password' /var/log/auth.log

Count the failures:

sudo grep 'Failed password' /var/log/auth.log | wc -l

Show source IPs ranked by number of failures:

sudo grep 'Failed password' /var/log/auth.log | awk '{print $(NF-3)}' | sort | uniq -c | sort -rn

Other useful grep options I practised:

Option	Meaning	Example
`-i`	Ignore case	`grep -i 'failed' auth.log`
`-v`	Invert match (exclude lines)	`grep -v 'CRON' syslog`

`-c`	Count matching lines	`grep -c 'Failed' auth.log`
`-n`	Show line numbers	`grep -n 'Failed' auth.log`
`-A 2 -B 2`	Show 2 lines after and before each match	`grep -A 2 -B 2 'Failed' auth.log`
`-E`	Extended regex (OR patterns)	`grep -E 'Failed|Invalid' auth.log`

[Screenshot 4 (key evidence): grep 'Failed' /var/log/auth.log showing failed-login results]

Findings: [Describe what you saw: how many failures, which usernames, which source IP, and the time window.]

Interpretation: A large number of failed passwords from a single IP address within a short time is the classic pattern of a brute-force attack. An analyst would note the source IP, the targeted usernames and whether any `Accepted password` line follows, which could indicate a successful compromise.

6. Monitoring Logs Live with tail -f

I ran the following in one terminal:

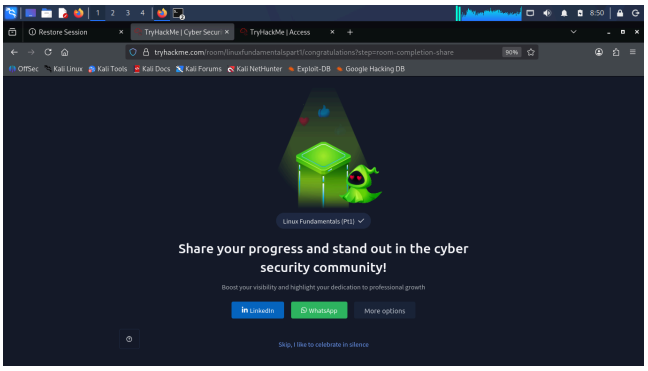
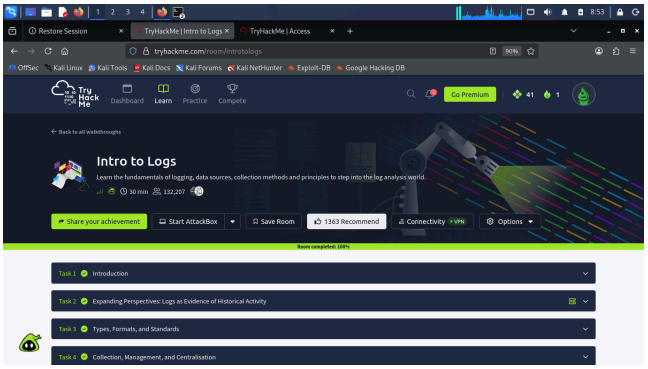
sudo tail -f /var/log/auth.log

In a second terminal I triggered a failed login, and the new line appeared immediately in the first terminal. This is how analysts watch for activity in real time during an incident or while testing detections. Pressing `Ctrl+C` stops the follow.

[Screenshot 5: tail -f showing a new failed-login line appearing live]

7. TryHackMe Rooms Completed

Room	Status	Proof

Linux Fundamentals Part 1	Completed	[Screenshot 6: room completion, username visible ]
Intro to Logs	Completed	[Screenshot 7: room completion ]

Key learning from the rooms: in every incident , theres always logs that help us triage alerts and investigate crimes what log sources and centralised logging matters because help use solve incidents

8. Challenges and Key Takeaways

Challenges: auth.log missing and resolved by installing rsyslog or using journalctl; permission denied until using sudo.]

Takeaways:

- Most security-relevant evidence on Linux lives in `/var/log`, with `auth.log` central for authentication events.
- `grep`, `less`, `tail` and simple pipes (`sort`, `uniq`, `wc`, `awk`) turn raw logs into useful findings quickly.
- `tail -f` supports live monitoring, while grep supports after-the-fact investigation.

- Repeated failed logins from one source are a key indicator of brute-force attempts, and this is a common alert source in a SOC.

9. Conclusion

This task built practical skills in Linux navigation and log analysis that I will use daily as a SOC analyst. I can now locate the key log files, search them for failed logins, count and rank the results, and monitor logs live, and I understand what those findings mean in a security context